

## 1. Alternate Printing (Even-Odd)

**Create two threads: one prints even numbers, another prints odd numbers (1–100) in sequence.**

**Code:**

```
class NumberPrinter {  
  
    int number = 1;  
  
    int limit = 100;  
  
    synchronized void printOdd() {  
  
        while (number <= limit) {  
  
            if (number % 2 == 0) {  
  
                try {  
  
                    wait();  
  
                } catch (InterruptedException e) {  
  
                    System.out.println(e);  
  
                }  
  
            } else {  
  
                System.out.print("Odd: " + number + " ");  
  
                number++;  
  
                notify();  
  
            }  
  
        }  
  
    }  
  
    synchronized void printEven() {  
  
        while (number <= limit) {  
  
            if (number % 2 != 0) {  
  
                try {  
  
                    wait();  
  
                }  
  
            }  
  
        }  
  
    }  
  
}
```

```

        } catch (InterruptedException e) {

            System.out.println(e);

        }

    } else {

        System.out.print("Even: " + number + " ");

        number++;

        notify();

    }

}

}

}

public static void main(String[] args) {

    NumberPrinter np = new NumberPrinter();

    Thread t1 = new Thread(() -> np.printOdd());

    Thread t2 = new Thread(() -> np.printEven());

    t1.start();

    t2.start();

}

}

```

### Output:

```

Odd: 1 Even: 2 Odd: 3 Even: 4 Odd: 5 Even: 6 Odd: 7 Even: 8 Odd: 9 Even: 10 Odd: 11 Even:
12 Odd: 13 Even: 14 Odd: 15 Even: 16 Odd: 17 Even: 18 Odd: 19 Even: 20 Odd: 21 Even:
22 Odd: 23 Even: 24 Odd: 25 Even: 26 Odd: 27 Even: 28 Odd: 29 Even: 30 Odd: 31 Even:
32 Odd: 33 Even: 34 Odd: 35 Even: 36 Odd: 37 Even: 38 Odd: 39 Even: 40 Odd: 41 Even:
42 Odd: 43 Even: 44 Odd: 45 Even: 46 Odd: 47 Even: 48 Odd: 49 Even: 50 Odd: 51 Even:
52 Odd: 53 Even: 54 Odd: 55 Even: 56 Odd: 57 Even: 58 Odd: 59 Even: 60 Odd: 61 Even:
62 Odd: 63 Even: 64 Odd: 65 Even: 66 Odd: 67 Even: 68 Odd: 69 Even: 70 Odd: 71 Even:
72 Odd: 73 Even: 74 Odd: 75 Even: 76 Odd: 77 Even: 78 Odd: 79 Even: 80 Odd: 81 Even:
82 Odd: 83 Even: 84 Odd: 85 Even: 86 Odd: 87 Even: 88 Odd: 89 Even: 90 Odd: 91 Even:
92 Odd: 93 Even: 94 Odd: 95 Even: 96 Odd: 97 Even: 98 Odd: 99 Even: 100

```

## 2. Print A B C in Sequence

- Three threads print A, B, C repeatedly in order (ABCABC...)

Code:

```
class ABCPrinter {

    int state = 1;

    int limit = 5;

    synchronized void printA() {

        for (int i = 0; i < limit; i++) {

            while (state != 1) {

                try {

                    wait();

                } catch (InterruptedException e) {

                    System.out.println(e);

                }

            }

            System.out.print("A ");

            state = 2;

            notifyAll();

        }

    }

    synchronized void printB() {

        for (int i = 0; i < limit; i++) {

            while (state != 2) {

                try {

                    wait();

                } catch (InterruptedException e) {

                    System.out.println(e);

                }

            }

        }

        System.out.print("B ");

    }

}
```

```

        state = 3;

        notifyAll();
    }
}

synchronized void printC() {

    for (int i = 0; i < limit; i++) {

        while (state != 3) {

            try {

                wait();

            } catch (InterruptedException e) {

                System.out.println(e);

            }

        }

        System.out.print("C ");

        state = 1;

        notifyAll();

    }

}

public static void main(String[] args) {

    ABCPrinter printer = new ABCPrinter();

    Thread t1 = new Thread(() -> printer.printA());

    Thread t2 = new Thread(() -> printer.printB());

    Thread t3 = new Thread(() -> printer.printC());

    t1.start();

    t2.start();

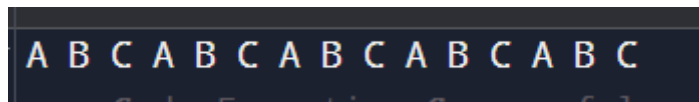
    t3.start();

}

}

```

**Output:**



```
A B C A B C A B C A B C A B C
```

### **3. Bank Account System**

- **Multiple threads deposit and withdraw from same account safely.**

**Code:**

```
public class BankSystem {  
  
    public static void main(String[] args) {  
  
        BankAccount account = new BankAccount();  
  
        Thread t1 = new Thread(() -> {  
  
            for (int i = 0; i < 3; i++) {  
  
                account.deposit(500);  
  
                try {  
  
                    Thread.sleep(1000);  
  
                } catch (Exception e) {}  
  
            }  
  
            }, "Depositor-1");  
  
        Thread t2 = new Thread(() -> {  
  
            for (int i = 0; i < 3; i++) {  
  
                account.withdraw(700);  
  
                try {  
  
                    Thread.sleep(1000);  
  
                } catch (Exception e) {}  
  
            }  
  
            }, "Withdrawer-1");  
  
        Thread t3 = new Thread(() -> {  
  
            for (int i = 0; i < 3; i++) {  
  
                account.deposit(300);
```

```

        try {

            Thread.sleep(1000);

        } catch (Exception e) {}

    }

    }, "Depositor-2");

    Thread t4 = new Thread(() -> {

        for (int i = 0; i < 3; i++) {

            account.withdraw(400);

            try {

                Thread.sleep(1000);

            } catch (Exception e) {}

        }

    }, "Withdrawer-2");

    t1.start();

    t2.start();

    t3.start();

    t4.start();

}

}

class BankAccount {

    private int balance = 1000;

    public synchronized void deposit(int amount) {

        balance += amount;

        System.out.println(Thread.currentThread().getN
ame() + " deposited " + amount + ", Balance: " +
        balance);

        notifyAll();

    }

}

```

```

public synchronized void withdraw(int amount) {

    while (balance < amount) {

        try {

            System.out.println(Thread.currentThread().getN
ame() +

                " waiting to withdraw " + amount);

            wait();

        } catch (InterruptedException e) {

            System.out.println(e);

        }

    }

    balance -= amount;

    System.out.println(Thread.currentThread().getN
ame() +

        " withdrew " + amount + ", Balance: " +

        balance);

}
}

```

Output:

```

Depositor-1 deposited 500, Balance: 1500
Withdrawer-2 withdrew 400, Balance: 1100
Depositor-2 deposited 300, Balance: 1400
Withdrawer-1 withdrew 700, Balance: 700
Depositor-1 deposited 500, Balance: 1200
Depositor-2 deposited 300, Balance: 1500
Withdrawer-2 withdrew 400, Balance: 1100
Withdrawer-1 withdrew 700, Balance: 400
Depositor-1 deposited 500, Balance: 900
Depositor-2 deposited 300, Balance: 1200
Withdrawer-1 withdrew 700, Balance: 500
Withdrawer-2 withdrew 400, Balance: 100

```

#### 4. Online Food Delivery System

- Multiple orders processed by limited delivery agents (thread pool).

Code:

```
import java.util.concurrent.*;

public class FoodDeliverySystem {

    public static void main(String[] args) {

        ExecutorService service = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 10; i++) {

            service.execute(new OrderTask(i));

        }

        service.shutdown();

    }

}

class OrderTask implements Runnable {

    private int orderId;

    public OrderTask(int orderId) {

        this.orderId = orderId;

    }

    public void run() {

        System.out.println(Thread.currentThread().getName() +

            " is processing Order " + orderId);

        try {

            Thread.sleep(2000);

        } catch (InterruptedException e) {

            System.out.println(e);

        }

        System.out.println(Thread.currentThread().getName() +
```



```
        " completed Order " + orderId);  
    }  
}
```

Output:

```
pool-1-thread-3 is processing Order 3  
pool-1-thread-1 is processing Order 1  
pool-1-thread-2 is processing Order 2  
pool-1-thread-2 completed Order 2  
pool-1-thread-1 completed Order 1  
pool-1-thread-1 is processing Order 5  
pool-1-thread-3 completed Order 3  
pool-1-thread-2 is processing Order 4  
pool-1-thread-3 is processing Order 6  
pool-1-thread-1 completed Order 5  
pool-1-thread-2 completed Order 4  
pool-1-thread-2 is processing Order 8  
pool-1-thread-1 is processing Order 7  
pool-1-thread-3 completed Order 6  
pool-1-thread-3 is processing Order 9  
pool-1-thread-2 completed Order 8  
pool-1-thread-2 is processing Order 10  
pool-1-thread-1 completed Order 7  
pool-1-thread-3 completed Order 9  
pool-1-thread-2 completed Order 10
```